

SOLID Principles Explained: SRP, OCP, and LSP

1. Single Responsibility Principle (SRP)

"Each class should have one reason to change"

What it means:

A class should have only **one job** or **one responsibility**. If a class has multiple reasons to change, it violates SRP.

✗ Bad Example (Violates SRP):

```
// This class has TOO MANY responsibilities
public class Student {
    private String name;
    private int age;
    private List<Double> grades;

    // Responsibility 1: Student data management
    public void setName(String name) { this.name = name; }
    public String getName() { return name; }

    // Responsibility 2: Grade calculations
    public double calculateGPA() {
        return grades.stream().mapToDouble(Double::doubleValue).average().orElse(0.0);
    }

    // Responsibility 3: Database operations
    public void saveToDatabase() {
        // Database saving logic
        System.out.println("Saving " + name + " to database...");
    }

    // Responsibility 4: Report generation
    public void generateTranscript() {
        System.out.println("=== TRANSCRIPT ===");
        System.out.println("Student: " + name);
        System.out.println("GPA: " + calculateGPA());
        // More report logic...
    }

    // Responsibility 5: Email notifications
    public void sendGradeNotification() {
        System.out.println("Sending grade notification to " + name);
        // Email sending logic...
    }
}
```

Problems with this approach:

- If database structure changes → Student class must change
- If GPA calculation rules change → Student class must change

- If email format changes → Student class must change
- If report format changes → Student class must change
- **One class has 5 different reasons to change!**



Good Example (Follows SRP):

```
// Each class has ONE responsibility

// Responsibility 1: Only student data
public class Student {
    private String name;
    private int age;
    private String studentId;
    private List<Double> grades;

    public Student(String name, int age, String studentId) {
        this.name = name;
        this.age = age;
        this.studentId = studentId;
        this.grades = new ArrayList<>();
    }

    // Only basic getters/setters and grade management
    public void addGrade(double grade) { grades.add(grade); }
    public List<Double> getGrades() { return new ArrayList<>(grades); }
    public String getName() { return name; }
    public String getStudentId() { return studentId; }
}

// Responsibility 2: Only GPA calculations
public class GPACalculator {
    public double calculateGPA(List<Double> grades) {
        if (grades.isEmpty()) return 0.0;
        return grades.stream().mapToDouble(Double::doubleValue).average().orElse(0.0);
    }

    public String getGradeLevel(double gpa) {
        if (gpa >= 3.5) return "A";
        else if (gpa >= 3.0) return "B";
        else if (gpa >= 2.5) return "C";
        else if (gpa >= 2.0) return "D";
        else return "F";
    }
}

// Responsibility 3: Only database operations
public class StudentRepository {
```

```

    public void save(Student student) {
        System.out.println("Saving student " + student.getName() + " to database...");
        // Database logic here
    }

    public Student findById(String studentId) {
        System.out.println("Finding student with ID: " + studentId);
        // Database query logic
        return null; // Simplified
    }
}

// Responsibility 4: Only report generation
public class TranscriptGenerator {
    private GPACalculator gpaCalculator;

    public TranscriptGenerator() {
        this.gpaCalculator = new GPACalculator();
    }

    public void generateTranscript(Student student) {
        double gpa = gpaCalculator.calculateGPA(student.getGrades());
        String gradeLevel = gpaCalculator.getGradeLevel(gpa);

        System.out.println("=== OFFICIAL TRANSCRIPT ===");
        System.out.println("Student: " + student.getName());
        System.out.println("ID: " + student.getStudentId());
        System.out.println("GPA: " + String.format("%.2f", gpa));
        System.out.println("Grade Level: " + gradeLevel);
    }
}

// Responsibility 5: Only email notifications
public class NotificationService {
    public void sendGradeNotification(Student student, double gpa) {
        System.out.println("📧 Sending grade notification to " + student.getName());
        System.out.println("Subject: Your GPA Update");
        System.out.println("Your current GPA is: " + String.format("%.2f", gpa));
    }
}

```

Benefits of SRP:

- **Easier to maintain** - each class has one clear purpose
- **Easier to test** - smaller, focused classes
- **More reusable** - GPACalculator can be used anywhere
- **Less coupling** - changes in one area don't affect others

2. Open/Closed Principle (OCP)

"Open for extension, closed for modification"

What it means:

You should be able to **add new functionality** without **changing existing code**. Use inheritance, interfaces, and polymorphism to extend behavior.

✗ Bad Example (Violates OCP):

```
// Every time we add a new shape, we must MODIFY this class
public class AreaCalculator {
    public double calculateArea(Object shape, String shapeType) {
        if (shapeType.equals("circle")) {
            Circle circle = (Circle) shape;
            return Math.PI * circle.getRadius() * circle.getRadius();
        }
        else if (shapeType.equals("rectangle")) {
            Rectangle rectangle = (Rectangle) shape;
            return rectangle.getWidth() * rectangle.getHeight();
        }
        else if (shapeType.equals("triangle")) {
            Triangle triangle = (Triangle) shape;
            return 0.5 * triangle.getBase() * triangle.getHeight();
        }
        // What if we want to add Pentagon? We must MODIFY this method!
        else {
            throw new IllegalArgumentException("Unknown shape type");
        }
    }
}

class Circle {
    private double radius;
    public Circle(double radius) { this.radius = radius; }
    public double getRadius() { return radius; }
}

class Rectangle {
    private double width, height;
    public Rectangle(double width, double height) {
        this.width = width; this.height = height;
    }
    public double getWidth() { return width; }
    public double getHeight() { return height; }
}
```

Problems:

- Adding new shapes requires **modifying** existing AreaCalculator code

- Risk of breaking existing functionality
- Violates OCP because the class is not "closed for modification"



Good Example (Follows OCP):

```
// Abstract base - defines contract
public abstract class Shape {
    public abstract double calculateArea();
    public abstract String getShapeName();
}

// Concrete implementations
public class Circle extends Shape {
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    @Override
    public double calculateArea() {
        return Math.PI * radius * radius;
    }

    @Override
    public String getShapeName() {
        return "Circle";
    }
}

public class Rectangle extends Shape {
    private double width, height;

    public Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }

    @Override
    public double calculateArea() {
        return width * height;
    }

    @Override
    public String getShapeName() {
        return "Rectangle";
    }
}
```

```

    }
}

public class Triangle extends Shape {
    private double base, height;

    public Triangle(double base, double height) {
        this.base = base;
        this.height = height;
    }

    @Override
    public double calculateArea() {
        return 0.5 * base * height;
    }

    @Override
    public String getShapeName() {
        return "Triangle";
    }
}

// This calculator is CLOSED for modification, OPEN for extension
public class AreaCalculator {
    public double calculateTotalArea(List<Shape> shapes) {
        return shapes.stream()
            .mapToDouble(Shape::calculateArea)
            .sum();
    }

    public void printShapeAreas(List<Shape> shapes) {
        for (Shape shape : shapes) {
            System.out.println(shape.getShapeName() + " area: " +
                String.format("%.2f", shape.calculateArea()));
        }
    }
}

// Adding new shape - NO modification to existing code needed!
public class Pentagon extends Shape {
    private double side;

    public Pentagon(double side) {

```

```

        this.side = side;
    }

    @Override
    public double calculateArea() {
        // Pentagon area formula
        return (Math.sqrt(25 + 10 * Math.sqrt(5)) / 4) * side * side;
    }

    @Override
    public String getShapeName() {
        return "Pentagon";
    }
}

// Usage example
public class ShapeDemo {
    public static void main(String[] args) {
        List<Shape> shapes = Arrays.asList(
            new Circle(5),
            new Rectangle(4, 6),
            new Triangle(3, 8),
            new Pentagon(4) // New shape works without changing AreaCalculator!
        );

        AreaCalculator calculator = new AreaCalculator();
        calculator.printShapeAreas(shapes);

        double totalArea = calculator.calculateTotalArea(shapes);
        System.out.println("Total area: " + String.format("%.2f", totalArea));
    }
}

```

Benefits of OCP:

- **Add new shapes** without changing existing code
- **Less risk** of breaking existing functionality
- **More maintainable** and scalable design
- **Follows polymorphism** principles

3. Liskov Substitution Principle (LSP)

"Subclasses should be substitutable for their base classes"

What it means:

If you have a function that works with a **parent class**, it should work correctly with **any subclass** without knowing which specific subclass it is.

✗ Bad Example (Violates LSP):

```
public class Bird {
    public void fly() {
        System.out.println("Flying through the sky!");
    }

    public void makeSound() {
        System.out.println("Chirp chirp!");
    }
}

public class Eagle extends Bird {
    @Override
    public void fly() {
        System.out.println("Eagle soaring high!");
    }

    @Override
    public void makeSound() {
        System.out.println("Screech!");
    }
}

// This violates LSP!
public class Penguin extends Bird {
    @Override
    public void fly() {
        // Penguins can't fly! This breaks the contract
        throw new UnsupportedOperationException("Penguins cannot fly!");
    }

    @Override
    public void makeSound() {
        System.out.println("Squawk!");
    }
}

// This code will break when Penguin is used
public class BirdHandler {
    public void makeBirdFly(Bird bird) {
        bird.fly(); // This will crash if bird is a Penguin!
    }
}
```

```

public static void main(String[] args) {
    List<Bird> birds = Arrays.asList(
        new Eagle(),
        new Penguin() // This will cause an exception!
    );

    BirdHandler handler = new BirdHandler();
    for (Bird bird : birds) {
        handler.makeBirdFly(bird); // Crashes on Penguin
    }
}

```

Problems:

- Penguin cannot be substituted for Bird without breaking the program
- The fly() method contract is violated
- Code that works with Bird fails with Penguin



Good Example (Follows LSP):

```
// Base class with only common behaviors
public abstract class Bird {
    protected String name;

    public Bird(String name) {
        this.name = name;
    }

    public abstract void makeSound();

    public void eat() {
        System.out.println(name + " is eating.");
    }

    public String getName() {
        return name;
    }
}

// Interface for birds that can fly
public interface Flyable {
    void fly();
    int getMaxAltitude();
}

// Interface for birds that can swim
public interface Swimmable {
    void swim();
    int getMaxDepth();
}

// Flying birds implement Flyable
public class Eagle extends Bird implements Flyable {
    public Eagle() {
        super("Eagle");
    }

    @Override
    public void makeSound() {
        System.out.println("Screech!");
    }
}
```

```

@Override
public void fly() {
    System.out.println(name + " soaring high in the sky!");
}

@Override
public int getMaxAltitude() {
    return 10000; // 10,000 feet
}
}

```

```

public class Sparrow extends Bird implements Flyable {
    public Sparrow() {
        super("Sparrow");
    }
}

```

```

@Override
public void makeSound() {
    System.out.println("Chirp chirp!");
}

```

```

@Override
public void fly() {
    System.out.println(name + " flying quickly!");
}

```

```

@Override
public int getMaxAltitude() {
    return 500; // 500 feet
}
}

```

```

// Swimming birds implement Swimmable
public class Penguin extends Bird implements Swimmable {
    public Penguin() {
        super("Penguin");
    }

    @Override
    public void makeSound() {
        System.out.println("Squawk!");
    }
}

```



```

@Override
public void swim() {
    System.out.println(name + " swimming gracefully underwater!");
}

@Override
public int getMaxDepth() {
    return 500; // 500 meters
}
}

// Some birds can both fly and swim
public class Duck extends Bird implements Flyable, Swimmable {
    public Duck() {
        super("Duck");
    }

    @Override
    public void makeSound() {
        System.out.println("Quack!");
    }

    @Override
    public void fly() {
        System.out.println(name + " flying over the water!");
    }

    @Override
    public int getMaxAltitude() {
        return 1000; // 1,000 feet
    }

    @Override
    public void swim() {
        System.out.println(name + " swimming on the surface!");
    }

    @Override
    public int getMaxDepth() {
        return 10; // 10 meters
    }
}

```

```

// Handlers work with appropriate interfaces
public class BirdActivityManager {

    // This method works with ANY Bird subclass
    public void feedBird(Bird bird) {
        System.out.println("Feeding " + bird.getName());
        bird.eat();
        bird.makeSound();
    }

    // This method only works with flying birds
    public void organizeFlightShow(List<Flyable> flyingBirds) {
        System.out.println("=== Flight Show ===");
        for (Flyable flyer : flyingBirds) {
            flyer.fly();
            System.out.println("Max altitude: " + flyer.getMaxAltitude() + " feet");
        }
    }

    // This method only works with swimming birds
    public void organizeSwimShow(List<Swimmable> swimmingBirds) {
        System.out.println("=== Swimming Show ===");
        for (Swimmable swimmer : swimmingBirds) {
            swimmer.swim();
            System.out.println("Max depth: " + swimmer.getMaxDepth() + " meters");
        }
    }

    public static void main(String[] args) {
        // All these are valid Bird substitutions
        List<Bird> allBirds = Arrays.asList(
            new Eagle(),
            new Sparrow(),
            new Penguin(),
            new Duck()
        );

        BirdActivityManager manager = new BirdActivityManager();

        // This works with ALL birds (LSP satisfied)
        System.out.println("=== Feeding Time ===");
        for (Bird bird : allBirds) {

```

```

        manager.feedBird(bird); // Works with any Bird subclass
    }

    // Separate activities for different capabilities
    List<Flyable> flyers = Arrays.asList(
        new Eagle(),
        new Sparrow(),
        new Duck()
    );
    manager.organizeFlightShow(flyers);

    List<Swimmable> swimmers = Arrays.asList(
        new Penguin(),
        new Duck()
    );
    manager.organizeSwimShow(swimmers);
}
}

```

Benefits of LSP:

- **Any Bird subclass** can be used wherever Bird is expected
- **No unexpected exceptions** or broken behavior
- **Interfaces separate capabilities** (flying, swimming) from identity (being a bird)
- **Code is more robust** and predictable

Real-World Example: Payment Processing System

Let's see all three principles working together:

```

// SRP: Each class has one responsibility
public class PaymentAmount {
    private double amount;
    private String currency;

    public PaymentAmount(double amount, String currency) {
        this.amount = amount;
        this.currency = currency;
    }

    public double getAmount() { return amount; }
    public String getCurrency() { return currency; }
}

// OCP & LSP: Payment methods can be extended without modifying existing code
public abstract class PaymentProcessor {
    public abstract boolean processPayment(PaymentAmount amount);
    public abstract String getPaymentMethodName();

    // Common validation - all subclasses inherit this
    protected boolean validateAmount(PaymentAmount amount) {
        return amount.getAmount() > 0;
    }
}

// Concrete implementations - can substitute for PaymentProcessor (LSP)
public class CreditCardProcessor extends PaymentProcessor {
    private String cardNumber;

    public CreditCardProcessor(String cardNumber) {
        this.cardNumber = cardNumber;
    }

    @Override
    public boolean processPayment(PaymentAmount amount) {
        if (!validateAmount(amount)) return false;

        System.out.println("Processing $" + amount.getAmount() +
                           " via Credit Card ending in " + cardNumber.substring(cardNumber.length() - 4));
        // Credit card processing logic
        return true;
    }
}

```

```

@Override
public String getPaymentMethodName() {
    return "Credit Card";
}
}

public class PayPalProcessor extends PaymentProcessor {
    private String email;

    public PayPalProcessor(String email) {
        this.email = email;
    }

    @Override
    public boolean processPayment(PaymentAmount amount) {
        if (!validateAmount(amount)) return false;

        System.out.println("Processing $" + amount.getAmount() +
            " via PayPal account: " + email);
        // PayPal processing logic
        return true;
    }

    @Override
    public String getPaymentMethodName() {
        return "PayPal";
    }
}

// SRP: Only handles payment coordination
public class PaymentService {
    public boolean processPayment(PaymentProcessor processor, PaymentAmount amount) {
        System.out.println("Starting payment processing with " + processor.getPaymentMethodName());

        boolean success = processor.processPayment(amount); // LSP in action

        if (success) {
            System.out.println("Payment successful!");
        } else {
            System.out.println("Payment failed!");
        }

        return success;
    }
}

```

```

    }
}

// Usage - demonstrates all principles
public class PaymentDemo {
    public static void main(String[] args) {
        PaymentService service = new PaymentService();
        PaymentAmount amount = new PaymentAmount(99.99, "USD");

        // LSP: Any PaymentProcessor subclass works the same way
        List<PaymentProcessor> processors = Arrays.asList(
            new CreditCardProcessor("1234567890123456"),
            new PayPalProcessor("user@example.com")
        );

        for (PaymentProcessor processor : processors) {
            service.processPayment(processor, amount); // Works with any processor
        }

        // OCP: We can add new payment methods without changing existing code
        // Just create a new class extending PaymentProcessor!
    }
}

```

Summary

Principle	Key Point	Benefit
SRP	One class, one responsibility	Easier to maintain and test
OCP	Extend behavior, don't modify existing code	Safer to add new features
LSP	Subclasses must work wherever parent works	More reliable inheritance

These principles work together to create **maintainable**, **extensible**, and **reliable** object-oriented code!